



TSwap Protocol Audit Report

Version 1.0

Cyfrin.io

June 17, 2026

Protocol Audit Report

Cyfrin.io

March 7, 2023

Prepared by: Cyfrin Lead Auditors: - xxxxxxxx

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
 - * [H-2] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing the users to receive the incorrect amount of tokens

- * [H-5] `TSwapPool::_swap` extra tokens give to users after every `swapCount` breaks the protocol invariant of $x * y = k$
- Medium
 - * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
 - [M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant
- Low
 - * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
- Informationals
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
 - * [I-2] Lacking zero address checks
 - * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`
 - * [I-4]: Event is missing `indexed` fields

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	2
Low	2
Info	9
Total	17

Findings

High

[H-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: Consider making the following change to the function.

```
1 function deposit(  
2     uint256 wethToDeposit,  
3     uint256 minimumLiquidityTokensToMint, // LP tokens -> if empty,  
        we can pick 100% (100% == 17 tokens)  
4     uint256 maximumPoolTokensToDeposit,  
5     uint64 deadline  
6 )  
7     external  
8 +     revertIfDeadlinePassed(deadline)  
9     revertIfZero(wethToDeposit)  
10    returns (uint256 liquidityTokensToMint)  
11    {
```

[H-2] Incorrect fee calculation in TSwapPool::getInputAmountBasedOnOutput causes protocol to take too many tokens from users, resulting in lost fees

Description: Get `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Recommended Mitigation:

```
1     function getInputAmountBasedOnOutput(  
        
```

```
2         uint256 outputAmount,  
3         uint256 inputReserves,  
4         uint256 outputReserves  
5     )  
6     public  
7     pure  
8     revertIfZero(outputAmount)  
9     revertIfZero(outputReserves)  
10    returns (uint256 inputAmount)  
11    {  
12 -        return ((inputReserves * outputAmount) * 10_000) / ((  
outputReserves - outputAmount) * 997);  
13 +        return ((inputReserves * outputAmount) * 1_000) / ((  
outputReserves - outputAmount) * 997);  
14    }
```

[H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction process, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a `swapExactOutput` looking for 1 WETH 1. inputToken = USDC 2. outputToken = WETH 3. outputAmount = 1 4. deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sends the protocol 10,000 USDC instead of the expected 1,000 USDC

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend upto a specific amount, and can predict how much they will spend on the protocol.

```
1     function swapExactOutput(  
2         IERC20 inputToken,  
3 +         uint256 maxInputAmount,  
4     .  
5     .  
6     .  
7         inputAmount = getInputAmountBasedOnOutput(outputAmount,  
            inputReserves, outputReserves);  
8 +         if(inputAmount > maxInputAmount){  
9 +             revert();
```

```
10 +     }
11     _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-4] TSwapPool::_sellPoolTokens mismatches input and output tokens causing the users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

Recommended Mitigation:

Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this also require changing the `sellPoolTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`)

```
1     function sellPoolTokens(
2         uint256 poolTokenAmount,
3 +     uint256 minWethToReceive
4     ) external returns (uint256 wethAmount) {
5 -     return swapExactOutput(i_poolToken, i_wethToken,
6 +     return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken
7         , minWethToReceive, uint64(block.timestamp));
8     }
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline. (MEV later)

[H-5] TSwapPool::_swap extra tokens give to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: - x : the balance of the pool token - y : balance of WETH - k : The constant product of two balances

This means that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the `k`. However, this is broken due to the extra incentive in the `_swap` function. Meaning that overtime the protocol funds will be drained.

The following block of code is responsible for the issue.

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1
5                                   _000_000_000_000_000_000);
6      }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put the protocol's core invariant is broken.

Proof of Concept:

1. A user swaps 10 times, and collects the extra incentive of `1_000_000_000_000_000_000` tokens
2. The user continues to swap until all the protocol funds are drained

Proof Of Code

Place the following into `TSwapPool.t.sol`

```
1      function testInvariantBroken() public {
2          vm.startPrank(LiquidityProvider);
3          weth.approve(address(pool), 100e18);
4          poolToken.approve(address(pool), 100e18);
5          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6          vm.stopPrank();
7
8          uint256 outputWeth = 1e17;
9
10         vm.startPrank(user);
11         poolToken.approve(address(pool), type(uint256).max);
12         poolToken.mint(user, 100e18);
13         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
14         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
15         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
16         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
17         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
18     }
```



```
18     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
19         timestamp));  
19     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
20         timestamp));  
20     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
21         timestamp));  
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
22         timestamp));  
22  
23     int256 startingY = int256(weth.balanceOf(address(pool)));  
24     int256 expectedDeltaY = int256(-1) * int256(outputWeth);  
25  
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
27         timestamp));  
27     vm.stopPrank();  
28  
29     uint256 endingY = weth.balanceOf(address(pool));  
30     int256 actualDeltaY = int256(endingY) - int256(startingY);  
31     assertEq(actualDeltaY, expectedDeltaY);  
32 }
```

Recommended Mitigation: Remove the extra incentive. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we should do with fees.

```
1 -     swap_count++;  
2 -     if (swap_count >= SWAP_COUNT_MAX) {  
3 -         swap_count = 0;  
4 -         outputToken.safeTransfer(msg.sender, 1_000_000_000_000);  
5 -     }
```

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence operations that liquidity to the pool might be executed at unexpected times, market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when the market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: Consider making following change to the function.

```
1     function deposit(  
2         uint256 wethToDeposit,  
3         uint256 minimumLiquidityTokensToMint,  
4         uint256 maximumPoolTokensToDeposit,  
5         uint64 deadline  
6     )  
7     external  
8 +     revertIfDeadlinePassed(deadline)  
9     revertIfZero(wethToDeposit)  
10    returns (uint256 liquidityTokensToMint)
```

[M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

/// finding...

Low

[L-1] TSwapPool::LiquidityAdded event has parameters out of order

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::addLiquidityMintAndTrans` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```
1 - emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);  
2 + emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by TSwapPool::swapExactInput results in incorrect return value given

Description The `swapExactInput` function is expected return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact The return value will always be 0, giving incorrect information to the caller.

Recommended Mitigation

```
1     {  
2         uint256 inputReserves = inputToken.balanceOf(address(this));
```

```
3         uint256 outputReserves = outputToken.balanceOf(address(this));
4
5 -         uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount,
6 +         inputReserves, outputReserves);
7         uint256 output = getOutputAmountBasedOnInput(inputAmount,
8 +         inputReserves, outputReserves);
9
10        if (outputAmount < minOutputAmount) {
11        -         revert TSwapPool__OutputTooLow(outputAmount,
12 +         minOutputAmount);
13        }
14
15        if (output < minOutputAmount) {
16        +         revert TSwapPool__OutputTooLow(outputAmount,
17 +         minOutputAmount);
18        }
19
20        _swap(inputToken, inputAmount, outputToken, outputAmount);
21        _swap(inputToken, inputAmount, outputToken, output);
22    }
```

Informationals

[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed

```
1 - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking zero address checks

```
1     constructor(address wethToken) {
2 +         if(wethToken == address(0)) {
3 +             revert();
4 +         }
5         i_wethToken = wethToken;
6     }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name()

```
1 -         string memory liquidityTokenName = string.concat("ts", IERC20(
2 +         tokenAddress).name());
3 +         string memory liquidityTokenName = string.concat("ts", IERC20(
4 +         tokenAddress).symbol());
```

[I-4]: Event is missing indexed fields

Index event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields). Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question. If there are fewer than three fields, all of the fields should be indexed.

4 Found Instances

- Found in src/PoolFactory.sol Line: 35

```
1 event PoolCreated(address tokenAddress, address poolAddress);
```

- Found in src/TSwapPool.sol Line: 43

```
1 event LiquidityAdded(address indexed liquidityProvider,  
    uint256 wethDeposited, uint256 poolTokensDeposited);
```

- Found in src/TSwapPool.sol Line: 44

```
1 event LiquidityRemoved(address indexed liquidityProvider,  
    uint256 wethWithdrawn, uint256 poolTokensWithdrawn);
```

- Found in src/TSwapPool.sol Line: 45

```
1 event Swap(address indexed swapper, IERC20 tokenIn, uint256  
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);
```